Teoría de Bases de Datos — Práctica 4: SQL

Licenciatura en Ciencias de la Computación · FCEIA–UNR

Esquema de trabajo. Todos los ejercicios se resuelven sobre la siguiente base de datos, cargada *completa* con `\i canon-postgresql.sql` —no sólo el núcleo—. Las diapositivas de la teoría muestran resultados de la instancia mínima, así que las mismas consultas devuelven ahí menos filas. Las claves primarias están subrayadas; `cuil` es clave alternativa de `persona`; `estudiante` y `docente` son subtipos de `persona`.

```
persona(dni, cuil, nombre, fecha_nacimiento)
          telefono(dni, telefono)
 estudiante(dni, carrera, promedio, activo)
          docente(dni, categoria)
   materia(codigo, titulo, dni_docente)
         cursa(dni, codigo, nota)
```

La instancia completa figura en el apéndice. Cada consulta debe resolverse primero en papel; el resultado esperado puede verificarse contra la instancia. En el laboratorio correspondiente, las mismas consultas se ejecutan sobre el motor.

Ejercicio 1. Consultas sobre una sola tabla

a) El nombre y la fecha de nacimiento de todas las personas registradas en la base de datos.

b) El dni y el promedio de los estudiantes de LCC con promedio mayor o igual que 8, ordenados por promedio descendente.

c) Las categorías de docente que existen en la base de datos, sin repeticiones.

d) El dni y la carrera de los estudiantes que no están activos.

e) El nombre de las personas cuyo apellido comienza con la letra G.

Ejercicio 2. Consultas sobre varias tablas

a) El nombre y la carrera de cada estudiante.

b) Por cada teléfono registrado: el nombre de la persona y el número.

c) El nombre de los estudiantes que cursan alguna materia dictada por Molina (dni 17222333).

d) El nombre de los estudiantes de LCC que obtuvieron alguna nota mayor que 8. Un mismo nombre puede aparecer repetido: explicar por qué ocurre y cómo se evita.

e) El título de las materias que ningún estudiante cursó.

f) Los pares de estudiantes distintos que comparten carrera, sin repetir pares. Informar los dos dni y la carrera.

g) Cada materia con el dni de quienes la cursan, incluidas las materias que nadie cursa, con `RIGHT JOIN`. Reescribirla con `LEFT JOIN` cambiando el orden de las tablas y comprobar que devuelve lo mismo.

h) Todos los pares (estudiante, materia) posibles, como planilla para la inscripción del cuatrimestre, con `CROSS JOIN`. ¿Cuántas filas tiene? ¿Cuántas de ésas ya están en `cursa`?

Ejercicio 3. Agregación y agrupamiento

a) La cantidad de cursados que tiene cada materia.

b) La nota promedio de cada materia, incluyendo las materias que nadie cursó. ¿Qué valor aparece para esas materias y por qué?

c) Las carreras con más de dos estudiantes.

d) Por cada docente, su nombre y la cantidad de materias que dicta.

Ejercicio 4. Subconsultas

a) El nombre de los estudiantes cuyo promedio supera al promedio de su propia carrera.

b) El nombre de las personas que no son estudiantes ni docentes.

c) El ítem anterior con `NOT EXISTS` y subconsultas correlacionadas, en lugar de `NOT IN`. ¿Qué cambia en la consulta y qué no cambia en el resultado?

d) El nombre y el promedio del o de los estudiantes con el promedio máximo.

Ejercicio 5. Modificación de datos

Los cambios de los ejercicios 5 y 6 son hipotéticos: salvo indicación en contrario, cada ítem parte del estado inicial, que se recupera ejecutando nuevamente `canon-postgresql.sql`. Ninguna clave foránea del esquema declara una acción referencial.

a) Registrar que Vega, Tomás (dni 41777205) cursó TBD con nota 7.

b) Intentar registrar nuevamente que López (dni 44876052) cursó TBD. ¿Qué responde el motor y qué restricción actúa?

c) Intentar registrar que López cursó la materia de código XX99. ¿Qué responde el motor y qué restricción actúa?

d) Marcar como inactivos a los estudiantes que no cursaron ninguna materia.

e) Intentar cambiar a 12 el promedio de Ibáñez (dni 43978415). ¿Qué responde el motor y qué restricción actúa?

Ejercicio 6. Eliminación de datos e integridad referencial

a) Eliminar el teléfono 341-777 de Paz (dni 40112233).

b) Eliminar todos los cursados con nota menor que 4. ¿Cuántas filas se eliminan?

c) Eliminar los cursados de los estudiantes que no están activos. ¿Cuántas filas se eliminan?

d) Intentar eliminar de `estudiante` a Ibáñez (dni 43978415). ¿Qué responde el motor? Si el esquema no declara ninguna acción referencial, ¿cuál está actuando por omisión?

e) Se quiere eliminar por completo a Quiroga (dni 45566778), que no cursó ninguna materia. ¿Por qué falla si se comienza por `persona`? Dar la secuencia de sentencias en el orden correcto.

f) Intentar eliminar de `persona` al docente Suárez (dni 27888999). Enumerar las tablas que, directa o indirectamente, referencian esa fila.

g) Dar la secuencia de sentencias que permite eliminar a Suárez con el esquema actual, reasignando primero sus materias al docente Molina (dni 17222333).

h) Si la clave foránea de `cursa` hacia `estudiante` se declarara con `ON DELETE CASCADE`, ¿qué produce eliminar a Ibáñez de `estudiante`? Indicar qué filas desaparecen, de qué tablas, y qué filas no se tocan.

i) Si la clave foránea de `cursa` hacia `materia` se declarara con `ON UPDATE CASCADE`, ¿qué produce cambiar el código TBD por BD02 en `materia`? ¿Cuántas filas de `cursa` cambian?

j) ¿Podría la clave foránea de `materia` hacia `docente` declararse con `ON DELETE SET NULL`? ¿Qué exigiría del esquema actual y qué significaría para una materia?

Ejercicio 7. Programación en la base de datos

Este ejercicio no pide escribir sintaxis de memoria: pide leer código y anticipar qué va a hacer. La sintaxis varía entre motores y se consulta; lo que no se consulta es el razonamiento.

7.1 Triggers: cuándo se disparan

Sobre la instancia inicial se agrega esta tabla y este trigger:

```
CREATE TABLE auditoria_cursado (
  id     integer   GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  dni    integer   NOT NULL,
  codigo varchar(4)   NOT NULL,
  nota   smallint,
  cuando timestamp NOT NULL);

CREATE FUNCTION auditar_cursado() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO auditoria_cursado (dni, codigo, nota, cuando)
  VALUES (NEW.dni, NEW.codigo, NEW.nota, now());
  RETURN NEW;
END; $$;

CREATE TRIGGER cursado_alta
AFTER INSERT ON cursa
FOR EACH ROW EXECUTE FUNCTION auditar_cursado();
```

Partiendo de `auditoria_cursado` vacía, se ejecutan en orden:

```
1)  INSERT INTO cursa VALUES (41220417,'PR01',6);
2)  INSERT INTO cursa VALUES (40112233,'AL01',5), (40112233,'TBD',7);
3)  UPDATE cursa SET nota=9 WHERE dni=41220417 AND codigo='PR01';
4)  INSERT INTO cursa VALUES (40112233,'XX99',5);
```

a) ¿Cuántas filas tiene `auditoria_cursado` después de cada una de las cuatro sentencias? Justificar cada respuesta.

b) La sentencia 4 falla. ¿Deja o no deja rastro en la auditoría? ¿Por qué?

c) La sentencia 3 modifica una nota que ya estaba auditada. ¿Queda registrado el cambio? Si la respuesta es no, ¿qué habría que agregar?

d) El mismo trigger se podría haber declarado `BEFORE INSERT` en lugar de `AFTER INSERT`. Las cuatro respuestas del ítem a) no cambian: explicar por qué. Indicar entonces qué sí es distinto entre los dos momentos, y dar un caso donde uno sirva y el otro no.

e) ¿Qué valor tiene `NEW.nota` en el trigger si la sentencia insertara un cursado sin nota, con el acta todavía abierta?

7.2 Procedimientos: qué hacer con quien no tiene notas

Las dos versiones recalculan el promedio de cada estudiante a partir de sus notas, y difieren en qué hacen con quien no tiene ninguna.

```
-- version A
UPDATE estudiante e
SET promedio = (SELECT ROUND(AVG(c.nota),1)
```

```
                FROM cursa c WHERE c.dni = e.dni);

-- version B
UPDATE estudiante e
SET promedio = COALESCE((SELECT ROUND(AVG(c.nota),1)
                         FROM cursa c WHERE c.dni = e.dni), 0);
```

a) Ejecutar las dos y comparar qué deja cada una en el promedio de Martín y Quiroga, que no tienen cursados.

b) ¿Cuál de los dos valores afirma algo falso sobre esos estudiantes, y qué afirma? Tener en cuenta el dominio de la nota, donde el 0 es una calificación con significado propio.

c) Una consulta que liste a los estudiantes con promedio mayor o igual que 8. ¿Aparecen esos dos después de la versión A? ¿Y después de la B? Explicar qué hace la comparación con un valor nulo.

7.3 Un cursor heredado: leerlo y reescribirlo

Este procedimiento aparece tal cual en un sistema en producción. No hay que escribir uno igual: hay que entenderlo y reemplazarlo.

```
CREATE PROCEDURE recalcular_promedios()
LANGUAGE plpgsql AS $$
DECLARE
  v_dni  integer;
  v_prom numeric(3,1);
  c CURSOR FOR SELECT dni FROM estudiante;
BEGIN
  OPEN c;
  LOOP
    FETCH c INTO v_dni;
    EXIT WHEN NOT FOUND;
    SELECT ROUND(AVG(nota),1) INTO v_prom
      FROM cursa WHERE dni = v_dni;
    UPDATE estudiante SET promedio = v_prom WHERE dni = v_dni;
  END LOOP;
  CLOSE c;
END; $$
```

a) Explicar en una sola oración qué hace, sin describir el mecanismo.

b) Reescribirlo como **una sola sentencia**, sin cursor ni bucle.

c) Contar, para cada versión, **cuántas sentencias SQL llega a ejecutar el motor** sobre la instancia de 12 estudiantes. Generalizar a n. Atención: la pregunta no es cuántas filas se leen —eso da parecido en las dos versiones y es justamente lo que despista.

d) El costo de la versión con cursor crece con n; el de la otra, no de la misma manera. Explicar de dónde sale esa diferencia, sabiendo que cada consulta que el procedimiento lanza vuelve a atravesar el motor entera: análisis, planificación y ejecución.

e) ¿En qué situación un cursor sería la opción razonable y no un defecto?

Ejercicio 8. Álgebra relacional

a) Expresar en álgebra relacional la consulta del ítem 2.a).

b) Dada la expresión $\pi_{nombre}\big(\sigma_{carrera='LM'}(persona \bowtie estudiante)\big)$, escribir la consulta SQL equivalente y su resultado sobre la instancia.

Apéndice: instancia de trabajo

persona

dni	nombre	fecha_nac.
17222333	Molina, Jorge	1965-11-05
22111444	Acosta, Graciela	1971-09-12
25666090	Benítez, Hugo	1976-02-28
27888999	Suárez, Marta	1980-06-17
33444555	Ríos, Claudia	1988-01-25
40111222	Martín, Ana Laura	1997-01-30
40112233	Paz, Carla	1997-02-08
41220417	Díaz, Juan	1998-09-23
41777205	Vega, Tomás	1998-04-04
42045662	Navarro, Bruno	1999-08-09
42890774	Gómez, Pedro	2000-03-02
43555010	Fernández, Lucía	2001-07-19
43978415	Ibáñez, Valentina	2001-12-01
44310589	Herrera, Diego	2002-05-14
44876052	López, Andrés	2003-04-12
45120336	Romero, Sofía	2003-10-27
45566778	Quiroga, Milena	2003-06-21

La columna `cuil` se omite por espacio.

estudiante

dni	carrera	promedio	activo
40111222	LCC	NULL	FALSE
40112233	LF	7.0	TRUE
41220417	LM	4.0	TRUE
41777205	LF	6.5	TRUE
42045662	IE	3.0	FALSE
42890774	LM	5.0	TRUE
43555010	LCC	8.0	TRUE
43978415	LCC	9.3	TRUE
44310589	LF	3.5	TRUE
44876052	LCC	8.5	TRUE
45120336	IE	9.0	TRUE
45566778	LM	NULL	TRUE

docente

dni	categoria
17222333	titular
22111444	adjunto
25666090	JTP
27888999	titular

materia

codigo	titulo	dni_docente
AL01	Álgebra Lineal	22111444
AM01	Análisis Matemático	25666090
TBD	Teoría de Bases de Datos	27888999
PR01	Programación	27888999
SO01	Sistemas Operativos	17222333

cursa

dni	codigo	nota
40112233	SO01	7
41220417	TBD	4
41777205	PR01	6
41777205	SO01	7
42045662	AL01	3
42890774	AL01	4
42890774	SO01	6
43555010	AL01	7
43555010	TBD	9
43555010	PR01	8
43978415	AL01	9
43978415	TBD	10
43978415	PR01	9
43978415	SO01	9
44310589	AL01	5
44310589	TBD	2
44876052	TBD	8
44876052	SO01	9
45120336	AL01	8
45120336	PR01	10

telefono

dni	telefono
33444555	341-9350
40112233	341-555
40112233	341-777
43555010	341-4001
43555010	341-4002